

DESIGN AND ANALYSIS OF ALGORITHMS

Performance Analysis

Sorting • Searching • Iterative & Recursive Factorial

Implementation: C
Performance measurement: clock()
Visualization: Python / Matplotlib

1. Project Overview

This project experimentally studies the performance and complexity of fundamental Design and Analysis of Algorithms (DAA) techniques. Two C programs are used: the first compares five sorting algorithms on the same random input, while the second compares Linear and Binary Search and also compares iterative and recursive factorial computation. Python is used only to visualize the measured C results and theoretical growth trends.

1.1 Objectives

- Implement and compare Bubble, Selection, Insertion, Merge and Quick Sort.
- Measure execution time on the same random dataset of 10,000 integers.
- Compare Linear Search $O(n)$ with Binary Search $O(\log n)$ using 100,000 repeated searches.
- Compare iterative and recursive factorial in terms of execution time and auxiliary space.
- Relate measured results to asymptotic time and space complexity.
- Present experimental results using clear tables, terminal evidence and graphs.

1.2 Experimental Setup

Parameter	Value
Language	C
Compiler / Environment	GCC in VS Code / PowerShell
Sorting input size	10,000 random integers
Search input size	10,000 random integers
Search test runs	100,000
Factorial input	$n = 10$
Factorial test runs	100,000
Timing method	C clock() / CLOCKS_PER_SEC
Visualization	Python with Matplotlib

Why 10,000 elements? A larger input was used so that timing differences were easier to observe on the test computer. Actual execution times can vary with hardware, compiler, random input and system load.

1.3 Project Structure

Program	Coverage
Program 1	Bubble, Selection, Insertion, Merge and Quick Sort performance comparison
Program 2	Linear vs Binary Search + Iterative vs Recursive Factorial
Python visualization	Six graphs based on the latest C results and theoretical complexity trends

2. Program 1 — Sorting Algorithm Performance Comparison

The first C program generates 10,000 random integers and copies the same unsorted dataset into separate arrays. This ensures that every sorting algorithm receives identical input. The execution time of each algorithm is measured independently.

2.1 Algorithms and Complexity

Algorithm	Best	Average	Worst	Auxiliary Space
Bubble Sort	$O(n^2)^*$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ average

*The submitted Bubble Sort implementation does not include an early-exit optimization, so its best case remains quadratic.

2.2 Working Principle

Bubble Sort: Repeatedly compares adjacent elements and swaps them when they are in the wrong order.

Selection Sort: Finds the minimum element in the unsorted region and places it at the next sorted position.

Insertion Sort: Builds a sorted prefix by inserting each new element into its correct position.

Merge Sort: Uses divide-and-conquer: split, recursively sort both halves, then merge.

Quick Sort: Partitions around a pivot and recursively sorts the two partitions.

2.3 Actual C Output

```

PS C:\C code> gcc daa.c
PS C:\C code> ./a.exe

=====
DAA - SORTING ALGORITHM PERFORMANCE COMPARISON
=====

Input Size : 10000
Input Type : Random Numbers

First 20 elements of original array:
[ 3762 3428 1880 4544 716 8651 406 562 5173 8008 4637 4608 9101 4694 4511 5770 5385 649 2831 8912 ... ]

First 20 elements after sorting:
[ 0 2 6 6 7 10 10 11 11 13 14 16 17 17 20 21 22 23 24 24 ... ]

=====
PERFORMANCE RESULTS
=====

+-----+-----+-----+
| Algorithm | Time (seconds) | Time Complexity |
+-----+-----+-----+
| Bubble Sort | 0.135000 |  $O(n^2)$  |
| Selection Sort | 0.051000 |  $O(n^2)$  |
| Insertion Sort | 0.037000 |  $O(n^2)^*$  |
| Merge Sort | 0.002000 |  $O(n \log n)$  |
| Quick Sort | 0.002000 |  $O(n \log n)^{**}$  |
+-----+-----+-----+

```

Figure 1. Sorting program terminal output — input sample and measured performance table.

```
* Insertion Sort average/worst case
** Quick Sort average case

=====
ANALYSIS
=====

Fastest Algorithm in this run : Merge Sort

Bubble Sort   :  $O(n^2)$ 
Selection Sort :  $O(n^2)$ 
Insertion Sort :  $O(n^2)$  average/worst
Merge Sort    :  $O(n \log n)$ 
Quick Sort     :  $O(n \log n)$  average

For large random input,  $O(n \log n)$  algorithms generally
perform better than  $O(n^2)$  algorithms.

Note: Actual execution times may vary depending on
the computer, compiler and generated random data.

=====
PS C:\C code> |
```

Figure 2. Sorting program terminal output — complexity analysis and conclusion.

2.4 Measured Sorting Results

Algorithm	Execution Time	Complexity	Observation
Bubble Sort	0.135 s	$O(n^2)$	Slowest in the measured run
Selection Sort	0.051 s	$O(n^2)$	Faster than Bubble Sort
Insertion Sort	0.037 s	$O(n^2)$ avg/worst	Best among quadratic sorts in this run
Merge Sort	0.002 s	$O(n \log n)$	Very fast; reported as fastest by the C run
Quick Sort	0.002 s	$O(n \log n)$ avg	Also very fast; displayed time tied after rounding

Merge Sort and Quick Sort both display 0.002 s because the terminal prints six decimal places. The program can still select Merge Sort as fastest if its unrounded internal timing is slightly smaller.

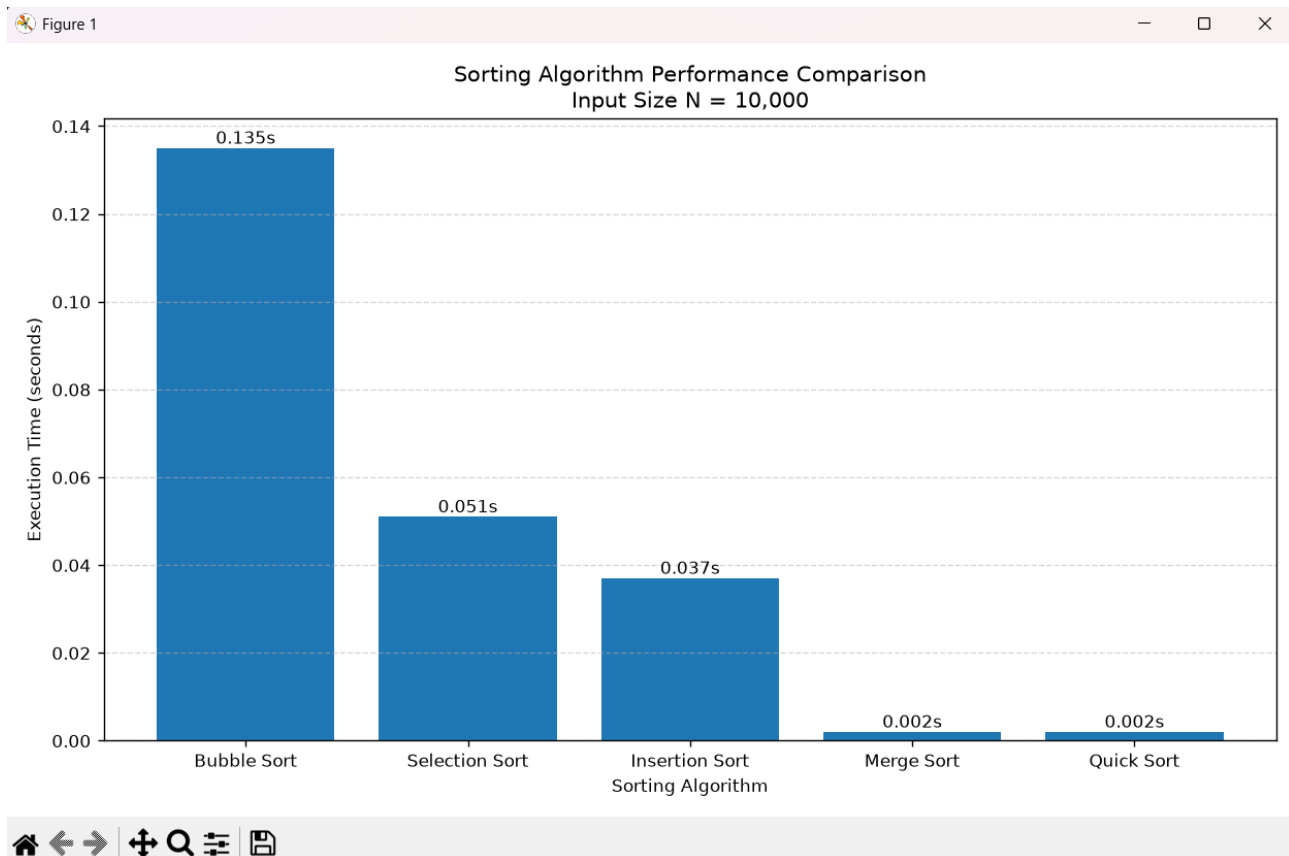


Figure 3. Sorting execution-time visualization based on the latest C run.

2.5 Interpretation

The experiment strongly separates the quadratic algorithms from the $O(n \log n)$ algorithms at $n = 10,000$. Bubble Sort requires the most time, while Merge Sort and Quick Sort finish in only a few milliseconds. This agrees with asymptotic analysis: as n grows, n^2 grows much faster than $n \log n$.

3. Program 2A — Linear Search vs Binary Search

The second C program first generates random data and creates a sorted copy using Quick Sort. Linear Search operates on the original array, while Binary Search operates on the sorted copy. A target that exists in the dataset is selected, and both search routines are repeated 100,000 times to obtain measurable execution times.

3.1 Complexity Comparison

Algorithm	Best Time	Average/Worst	Space	Requirement
Linear Search	$O(1)$	$O(n)$	$O(1)$	No sorting required
Binary Search	$O(1)$	$O(\log n)$	$O(1)$	Input must be sorted

3.2 Actual C Output

```

PS C:\C code> gcc daa1.c
PS C:\C code> ./a.exe

=====
DAA - LINEAR SEARCH vs BINARY SEARCH
=====

Input Size : 10000
Target     : 3759
Test Runs  : 100000

=====
SEARCH RESULTS
=====

+-----+-----+-----+-----+
| Algorithm | Found Index | Time (sec) | Steps |
+-----+-----+-----+-----+
| Linear Search | 5020 | 0.387000 | 5021 |
| Binary Search | 4292 | 0.004000 | 12 |
+-----+-----+-----+-----+

Search Complexity:
Linear Search:
Time :  $O(n)$ 
Space :  $O(1)$ 

Binary Search:
Time :  $O(\log n)$ 

```

Figure 4. Search program terminal output — measured time and comparison steps.

3.3 Measured Search Results

Metric	Linear Search	Binary Search
Execution time (100,000 runs)	0.387 s	0.004 s
Comparisons / steps	5021	12
Measured time ratio	$\approx 96.75\times$	Reference
Time complexity	$O(n)$	$O(\log n)$
Auxiliary space	$O(1)$	$O(1)$

In this particular run, Binary Search completed the repeated searches in approximately 1/96.75 of the measured Linear Search time. The comparison count is even more striking: 12 steps versus 5021 steps for the selected target.

3.4 Search Visualizations

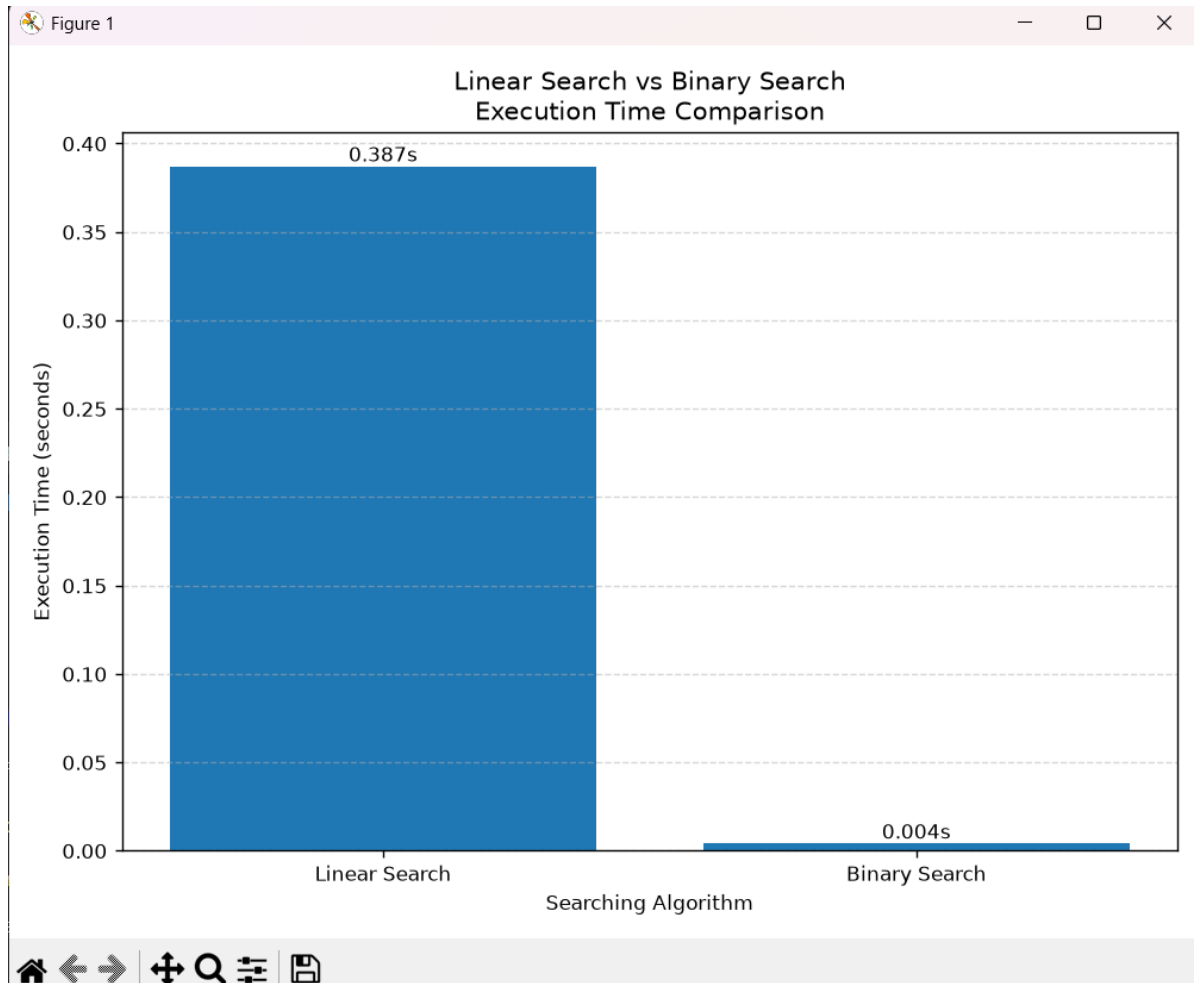


Figure 5. Measured execution time: Linear Search vs Binary Search.

Figure 1

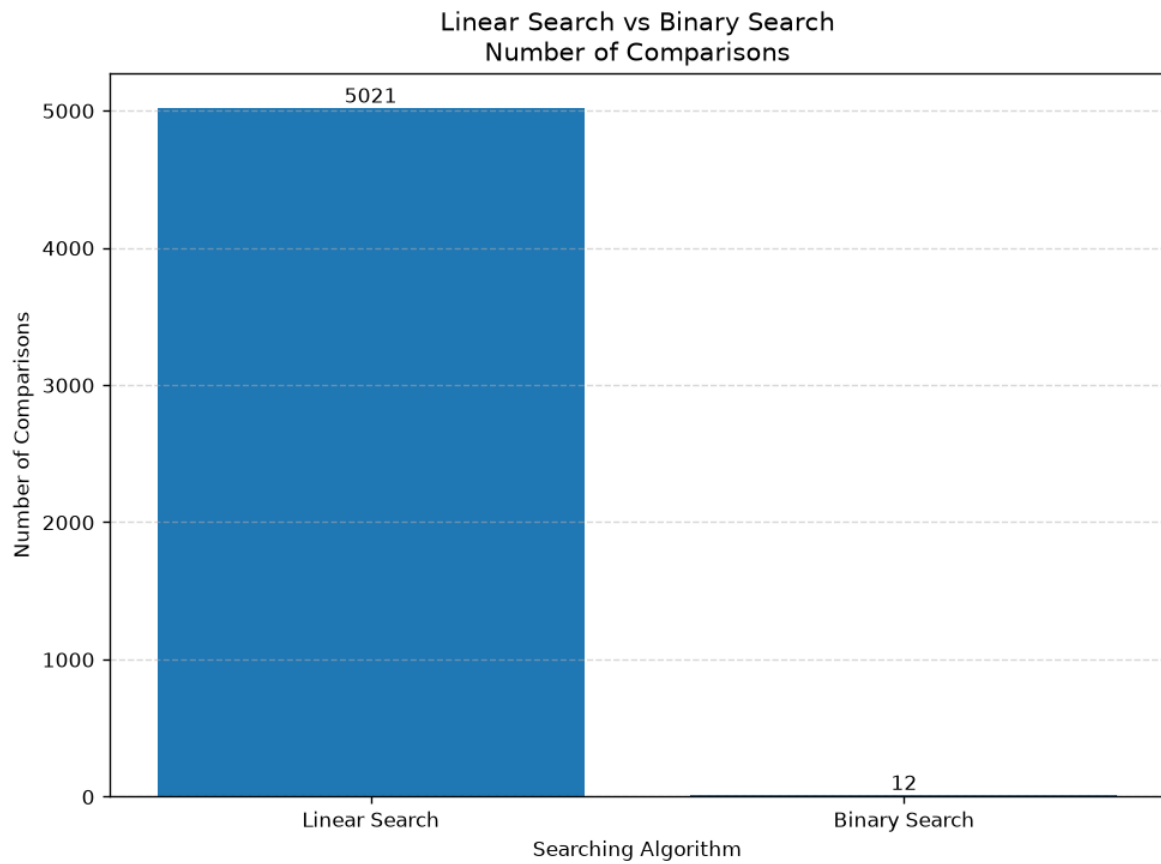


Figure 6. Measured number of comparisons for the selected target.

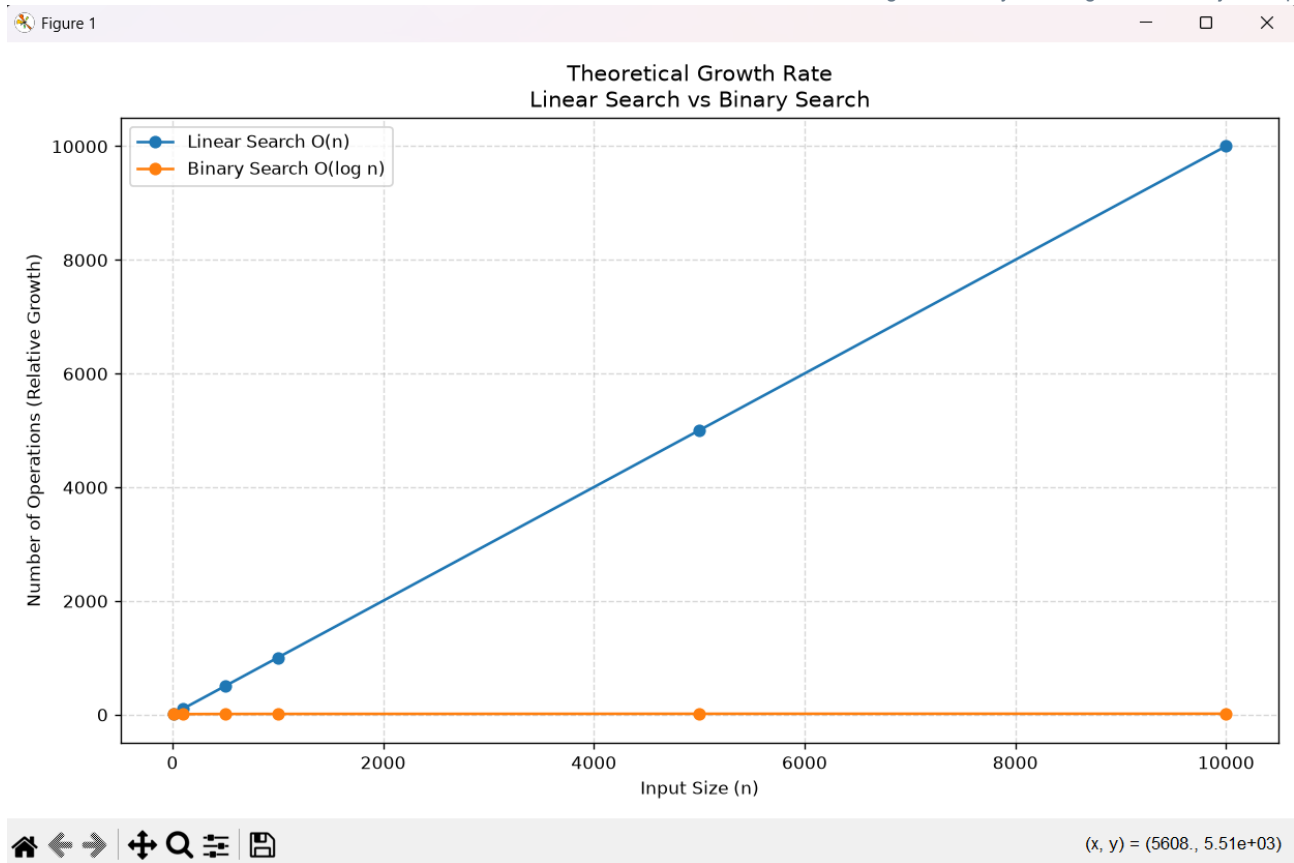


Figure 7. Theoretical growth: $O(n)$ Linear Search vs $O(\log n)$ Binary Search.

3.5 Interpretation

Linear Search may need to inspect a large fraction of the array, so its work grows directly with n . Binary Search repeatedly halves the remaining search interval, so the number of required comparisons grows logarithmically. For $n = 10,000$, $\log_2(n)$ is only about 13.29, which is consistent with the observed 12 comparison steps in this run.

Binary Search is faster only after the data is sorted. If a dataset is unsorted and only one search is required, the cost of sorting should also be considered. For repeated searches on a sorted dataset, Binary Search is highly efficient.

4. Program 2B — Iterative vs Recursive Factorial

The factorial experiment calculates $n!$ using two methods. The iterative version uses a loop and a single accumulator. The recursive version calls itself with $n - 1$ until reaching the base case. For the experiment, $n = 10$ and each method is executed 100,000 times.

4.1 Algorithm Analysis

Method	Time Complexity	Auxiliary Space	Reason
Iterative	$O(n)$	$O(1)$	One loop; constant number of variables
Recursive	$O(n)$	$O(n)$	n nested function calls occupy the call stack

4.2 Actual C Output

```

=====
ANALYSIS
=====
SEARCHING:
Linear Search:
- Time Complexity : O(n)
- Space Complexity : O(1)

Binary Search:
- Time Complexity : O(log n)
- Space Complexity : O(1)
- Requires sorted input

FACTORIAL:
Iterative Factorial:
- Time complexity : O(n)
- Space Complexity : O(1)

Recursive Factorial:
- Time Complexity : O(n)
- Space Complexity : O(n)
- Uses recursive call stack

=====
CONCLUSION
=====

Binary Search is more scalable than Linear Search
for large sorted datasets.

```

Figure 8. Factorial terminal output — $n = 10$, result and timing table.

```

=====
CONCLUSION
=====

Binary Search is more scalable than Linear Search
for large sorted datasets.

Both factorial methods have O(n) time complexity,
but the iterative method uses less extra memory.

=====
PS C:\C code>

```

Figure 9. Program conclusion shown in the terminal.

4.3 Measured Factorial Results

Metric	Iterative	Recursive
--------	-----------	-----------

Input	10	10
Result	3,628,800	3,628,800
Test runs	100,000	100,000
Execution time	0.003 s	0.003 s
Time complexity	$O(n)$	$O(n)$
Auxiliary space	$O(1)$	$O(n)$

4.4 Factorial Visualizations

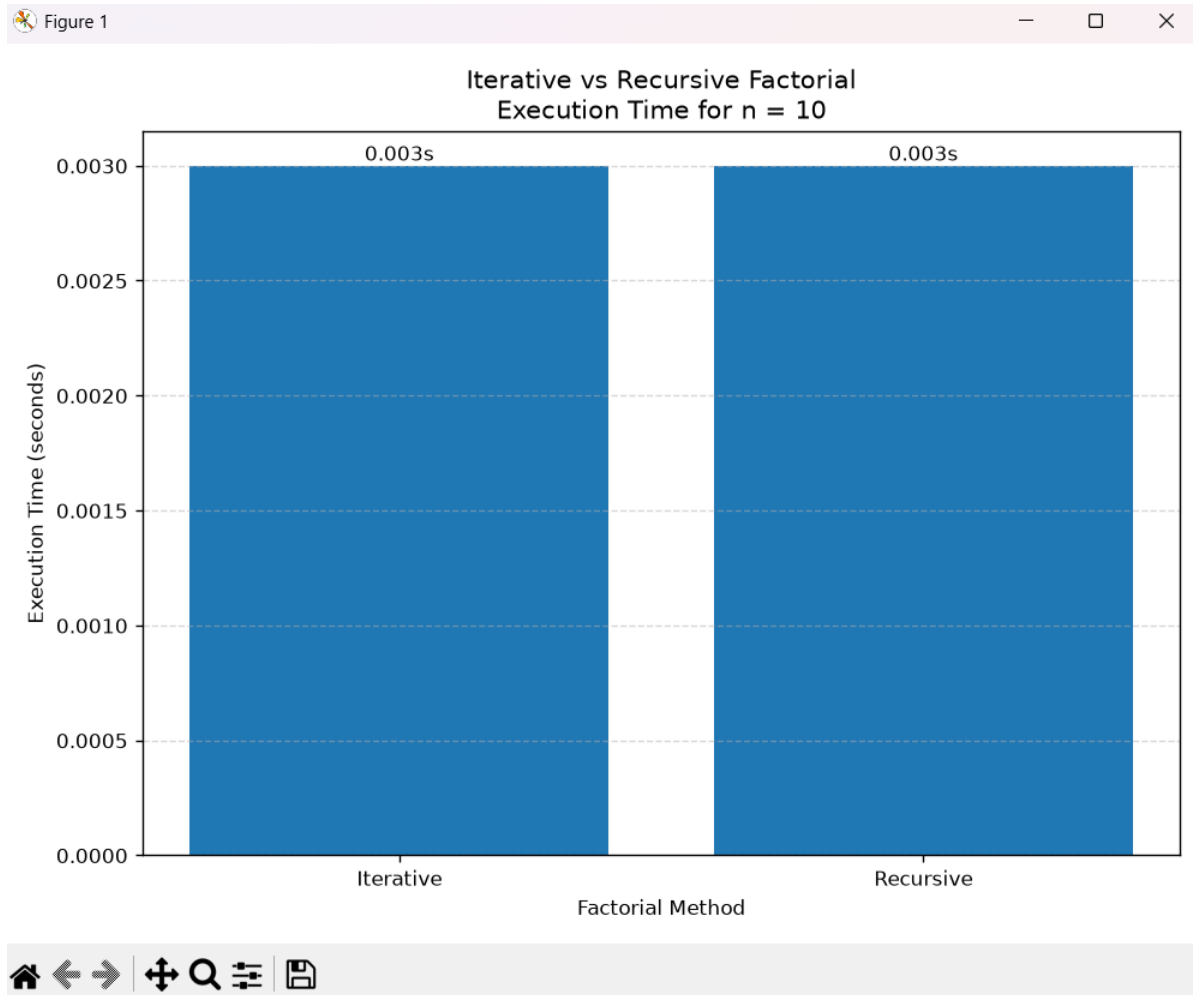


Figure 10. Iterative vs Recursive Factorial execution time for $n = 10$.

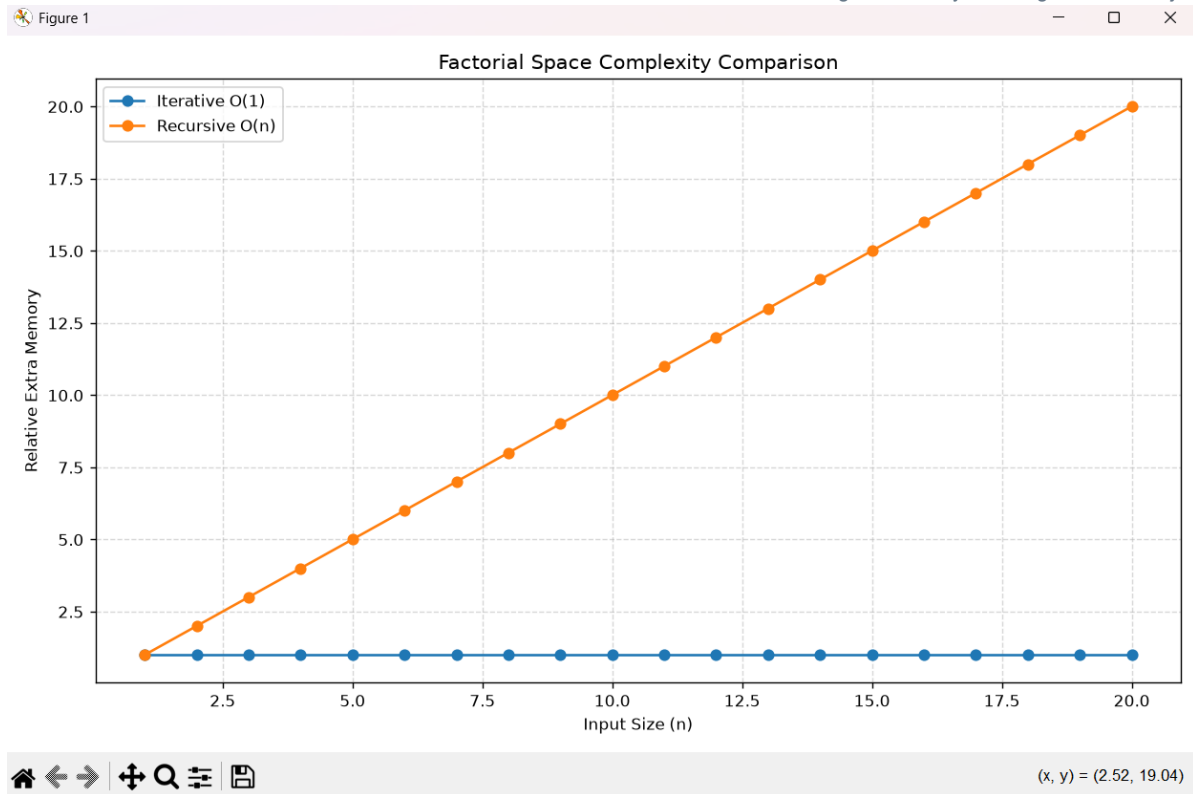


Figure 11. Theoretical auxiliary-space growth: $O(1)$ vs $O(n)$.

4.5 Interpretation

For $n = 10$, the measured execution times are identical at the displayed precision, so timing alone does not distinguish the two approaches in this run. The more important algorithmic difference is memory usage. The iterative method keeps constant auxiliary state, while recursion creates a call-stack frame for each level. Therefore, the iterative approach is more space-efficient and avoids recursion-depth overhead.

5. Consolidated Results and Discussion

5.1 Key Experimental Findings

- Quick Sort and Merge Sort were dramatically faster than the $O(n^2)$ sorting algorithms at $n = 10,000$.
- Bubble Sort was the slowest sorting method in the measured run.
- Binary Search required only 12 comparisons for the chosen target, compared with 5021 for Linear Search.
- Binary Search measured 0.004 s versus 0.387 s for Linear Search across 100,000 runs.
- Both factorial methods produced the correct value $10! = 3,628,800$.
- Iterative and recursive factorial had equal displayed timing in this small-input experiment, but their space complexities differ: $O(1)$ versus $O(n)$.

5.2 Measured vs Theoretical Behavior

Topic	Measured Evidence	Theoretical Explanation
Sorting	Merge/Quick ≈ 0.002 s; quadratic sorts 0.037–0.135 s	$n \log n$ grows much slower than n^2
Searching	Binary 0.004 s / 12 steps; Linear 0.387 s / 5021 steps	Repeated halving gives $O(\log n)$ rather than $O(n)$
Factorial time	Both 0.003 s	Both perform work proportional to n
Factorial space	Not measured in bytes; visualized theoretically	Iteration is $O(1)$; recursion uses $O(n)$ stack frames

5.3 Limitations

- `clock()` has finite timing resolution, so very fast operations may round to the same displayed value.
- Random input changes between runs, which can affect exact timings and Quick Sort partition behavior.
- Execution time depends on CPU, compiler optimization, operating-system load and background processes.
- The search timing excludes the one-time cost of sorting the array before repeated Binary Search operations.
- The factorial space graph is a theoretical relative-memory visualization, not a direct byte-level memory measurement.

5.4 Final Conclusion

The project demonstrates why asymptotic analysis is useful in practical algorithm selection. For large random sorting inputs, $O(n \log n)$ algorithms substantially outperform $O(n^2)$ methods. For repeated searching on sorted data, Binary Search provides a major reduction in both comparisons and measured execution time compared with Linear Search. For factorial computation, iterative and recursive methods have the same $O(n)$ time complexity, but the iterative method is more memory-efficient because it uses $O(1)$ auxiliary space instead of $O(n)$ recursive stack space. Overall, the experimental results are consistent with the expected DAA complexity models.

6.1 Tools Used

- C / GCC — algorithm implementation and execution-time measurement
- Visual Studio Code / PowerShell — compilation and execution
- Python / Matplotlib — visualization of experimental and theoretical results
- Microsoft Word — final project documentation